



TITAN ITCH & GLIMPSE Protocol Specifications

Release 1.4
2016-05-10

Singapore Exchange

Disclaimer

The material provided here is for information purposes only.

While reasonable care has been taken to ensure its accuracy, the information is furnished with no warranty as to the accuracy or completeness of its contents and on condition that any changes, omissions or errors shall not be made the basis for any claim, demand or cause for action.

SGX reserves the right to change any material in this document without notice.

This information is given to the recipient organizations in strict confidence, and accordingly, is to be kept confidential and is not to be disclosed to any person except officers and employees or third party developers/subcontractors of recipient organizations to the extent that they have a need to know.

© Singapore Exchange Limited.

All Rights Reserved. No part of this publication may be reproduced or transmitted, in any form or by any means whether, electronic, mechanical, or otherwise without the written permission of the copyright holder.

The written permission of the copyright holder must also be obtained before any part of this publication is stored in a retrieval system of any nature.

Applications for the copyright holder's written permission to reproduce, transmit or store in a retrieval system any part of this publication should be addressed to the publisher.

Market Access
Singapore Exchange Limited
2 Shenton Way
#19-00 SGX Centre 1
Singapore 068804

Facsimile: (65) 6532 0834
Email: access@sgx.com

Document History

Version	Version Date	Description and Reason for Change
1.0	2015-04-29	Initial Version
1.1	2015-05-28	Project Titan Update - Draft
1.2	2015-07-30	Project Titan Update - Comments
1.2	2015-08-04	Project Titan - Final
1.3	2015-12-09	Removed TMC and Hidden volume orders
1.4	2016-05-10	Updated the message Header

Contents

1.	Overview	1
1.1.	Purpose	1
1.2.	Intended Audience	1
1.3.	Change Control	1
1.4.	Change Implementation	1
1.5.	Restrictions on Software Distribution	2
1.6.	Contacting Support	2
1.7.	Reference & Glossary	2
2.	Introduction	4
2.1.	ITCH	4
2.2.	Glimpse	4
2.2.1.	ITCH/Glimpse Architecture	5
3.	ITCH	6
3.1.	Overview	6
3.2.	MoldUDP64 Protocol	6
3.2.1.	Number Fields	7
3.2.2.	Terms	7
3.2.3.	Downstream Packet	7
3.2.4.	Receiver Example	10
3.3.	ITCH Protocol	11
3.3.1.	Data Types	11
3.3.2.	Message Formats	12
3.3.3.	Reference Data Messages	13
3.3.4.	Event & State Change Messages	17
3.3.5.	Market by Order Messages	18
3.3.6.	Trade Messages	23
3.3.7.	Auction Messages	24
3.3.8.	Building an Order Book View	25
3.3.9.	Building a Trade Ticker	25
4.	Glimpse	26
4.1.	Overview	26
4.2.	SoupBinTCP Protocol	26
4.2.1.	SoupBinTCP Packet Types	29
4.2.2.	Data Types	33
4.2.3.	Glimpse Messages	33

5.	Configuration & Infrastructure.....	34
6.	Appendix 1 - ITCH Scenario Examples.....	35
6.1.	Disclosed Orders, Continuous Trading.....	35
6.1.1.	New Order Entry, Amend & Delete.....	35
6.1.2.	New Order Fully Trades with Existing Order(s).....	36
6.1.3.	New Order Partially Trades with Existing Order(s).....	36
6.1.4.	Existing Order is Amended and Immediately Fully Trades with Opposing Order.....	36
6.1.5.	Existing Order is Amended and Immediately Partially Trades with Opposing Order.....	37
6.2.	Disclosed Order Execution, Auctions.....	38
6.2.1.	Orders Trade in Auction.....	38
6.3.	Instrument Creation.....	39
6.3.1.	Standard Instrument Creation.....	39
6.4.	Miscellaneous Tasks.....	39
6.4.1.	Customer to Customer Order Transfer.....	39

List Of Figures

Figure 2.1	ITCH/Glimpse Architecture.....	5
Figure 3.1	Client Receiver Flowchart.....	10
Figure 4.1	SoupBinTCP Protocol Flow.....	27

1. Overview

This is the Protocol Specifications Manual for the ITCH/Glimpse products utilised for accessing SGX Market information.

1.1. Purpose

This Manual provides information that allows developers to develop user applications to interface with the ITCH/Glimpse application to access SGX Market Information.

1.2. Intended Audience

This Manual is written for information technology personnel of Members and independent software vendors (ISVs).

The target audience is systems designers and programmers as they will be responsible for developing applications to use the ITCH/Glimpse to interact with SGX.

1.3. Change Control

This Manual is available from SGX.

Updates to this manual will be issued periodically to all Members and ISVs.

Changes to the ITCH/Glimpse which require changes to the Manual will be notified to Members and ISV's before they are implemented. It is the responsibility of Members and ISVs to evaluate the changes to determine whether they need to modify their developed applications.

SGX reserves the right at all times to effect immediate changes without notice if those changes are required in order to maintain an orderly market.

1.4. Change Implementation

Changes to ITCH/Glimpse messages and values will be implemented by releasing a new software version on the implementation date.

Users and developers will be required to cut over to the new version on the implementation date, and the old version will not be supported after that date.

New versions of the ITCH/Glimpse messages and values will be available in OIT Test Centre prior to release so users can perform their own user application testing.

Any changes or upgrades to user's systems interfacing ITCH/Glimpse would require their new version of software to be certified in the Conformance test.

1.5. Restrictions on Software Distribution

There are restrictions on the distribution of the ITCH/Glimpse software covered in the sublicensing agreement.

1.6. Contacting Support

For technical enquiries and assistance with issues encountered during development and testing, you can contact the Open Interface Support (OIT) Support Team. This team also allocates test periods to developers and users.

Development :

Email: derivatives_dev@sgx.com

Telephone: (65) 6236 8400 -> Press 1 -> Press 1

Conformance:

Email: derivatives_conform@sgx.com

1.7. Reference & Glossery

This section provides a list of specific terminologies used in the document. It is also used as an acronym reference table, with the terms listed in alphabetical order for ease of use.

Term	Definition
Allocation	This is a booking of trades into a client's trading account using either average or executed pricing. A contract is likely to be issued by the Trading Member to its client for each allocation
SGX-DX	SGX Derivatives Clearing
Clearing Member	Corporation granted or admitted to be a Member of SGX-DC pursuant to the Clearing Rules and shall include a General Clearing Member and a Bank Clearing Member
Developer	Refers to both Members and ISVs who use the ITCH/Glimpse when developing applications to access the SGX Market information
Event Controls	Order Book State messages to inform receivers of changes
ISO	International Organization for Standardization
ISV	Independent Software Vendor
Member	A Member of SGX-DT or SGX-DC
Message	An atomic piece of information carried by the MoldUDP64 protocol
Order level data (MBO)	A series of messages to track the life of a customer order, including executions
Participant	Trading Member and Clearing Member
Reference Data Order Book	Directory messages to provide basic security data
Session	A Session is a sequence of one or more messages
SGX	Singapore Exchange Limited
ITCH/Glimpse	The application programmers interface used by developers to send and receive information from ITCH/Glimpse

Overview

Tick Size Table Entry	Price Tick determines the premiums allowed in an order; how many decimals that are allowed and the smallest steps (units) in which the premium can be quoted
Trade	A side of a deal executed between two parties for clearing. Each trade is booked into one account

Term	Definition
Trade Allocation	See 'Allocation'
Trading Member	An entity that has been approved as Member of SGX-DT and SGX-DC Rules
Trade messages	Messages to reflect a match of a non-displayable order within the system
UDP	User Datagram Protocol
User(s)	Refers to the member firm that uses a user application structured on the ITCH/Glimpse to access the SGX Market information

2. Introduction

2.1. ITCH

ITCH is a direct data feed product for accessing SGX Market information, and is designed to display all public orders/trades that occur on the auto-matched market(s).

The following Market information data elements are featured:

Data Elements	Sub-category Data Messages
Reference Data	Order Book Directory
	Combination Order Book Directory
	Tick Size Table Entry
Event Controls	System Event
	Order Book State
Market Order Level Data	Add Order
	Modify Order
	Order Executed
	Order Executed with Price
	Order Replace
	Order Delete
Trade	Trade
Auction	Equilibrium Price Update

Note: ITCH provides an Order Book view:

- It cannot be assumed that the mechanisms of the matching logic can be derived from observing the ITCH feed
- ITCH cannot be used to manage private orders
- ITCH does not provide privately negotiated trades reported to the marketplace

2.2. Glimpse

Glimpse, a compliment to the ITCH real time product, is a point to point data feed connection that provides direct data feed customers with a snapshot of the current state of Order Books within the SGX System. At the end of the Glimpse snapshot a sequence number is provided that be used to connect and sync up with the real-time ITCH feed.

2.2.1. ITCH/Glimpse Architecture

The following diagram illustrates the architectural layout from the ITCH/Glimpse gateways to the customer user servers.

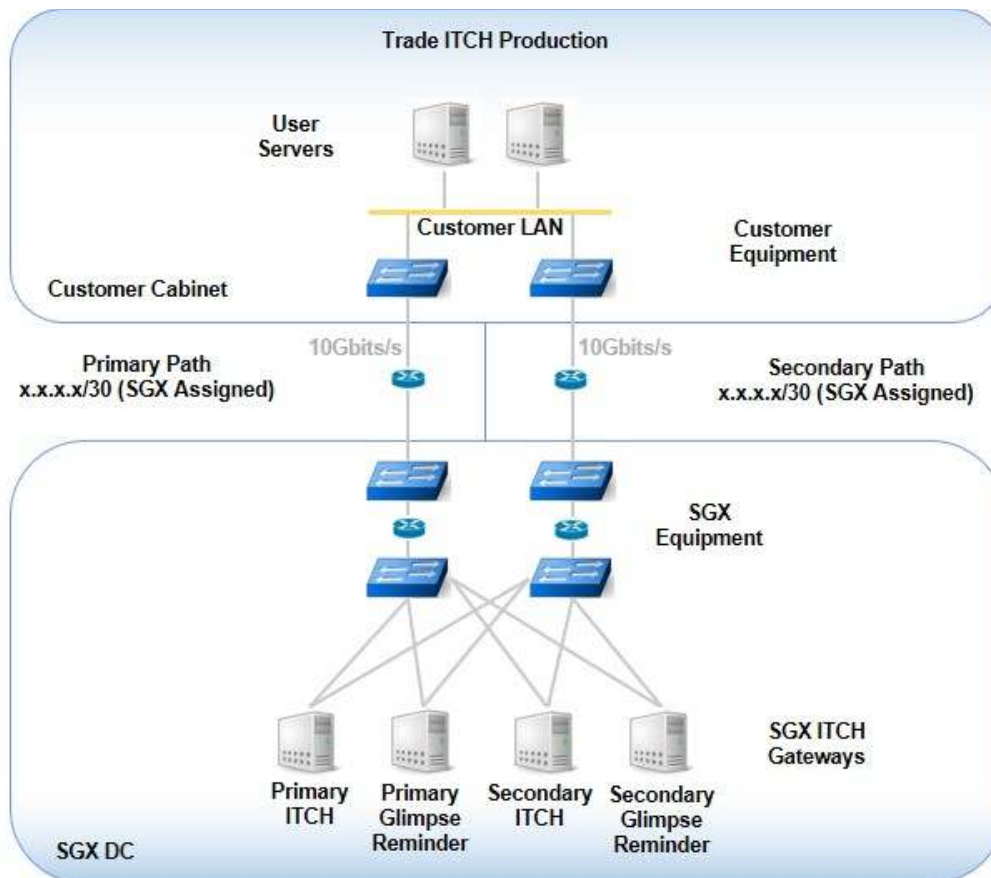


Figure 2.1 ITCH/Glimpse Architecture

3. ITCH

3.1. Overview

ITCH enables the tracking of the status of an order from the time it is first entered until the time it is either executed or cancelled and is intended for information exchange only.

Active trading is supported by the Titan OUCH protocol.

The ITCH protocol provides:

- Improved latency stability (reduced jitter, more so in ITCH)
- Multicast stream of Order Book changes
- Full order detail, meaning every quote and every order in every identified orderbook
- Trade data for all SGX orderbooks
- Access to all asset classes available in SGX
- Access to security status messages
- Access to basic security data including ISIN code, financial product and tick size
- Provides Time-stamping to the nano-second

The ITCH feed is made up of a series of sequenced messages. Each message is variable in length based on the message type. The messages are binary encoded using MoldUDP64 protocol.

3.2. MoldUDP64 Protocol

MoldUDP64 is a networking protocol that allows efficient and scalable transmission of data messages in a “one transmitter to many listeners” scenario.

MoldUDP64 is a lightweight protocol layer built on top of UDP that provides a mechanism for listeners to detect and re-request missed packets.

In MoldUDP64, each outbound packet is transmitted only once regardless of the number of listeners. Multiple messages may also be aggregated into a single network packet to reduce network traffic.

MoldUDP64 transmitters send downstream packets via UDP multicast to transport the normal data stream sent to the listeners. These packets are also sent via UDP unicast in response to a Request message submitted by a listener. MoldUDP64 clients can send these Request messages to request the retransmission of any desired packets from the data stream.

The MoldUDP64 server will transmit on a well-known multicast group for each type of downstream MoldUDP64 datastream on a network. The listeners must subscribe to this multicast group to receive the downstream data. One or more Re-request Servers may also be deployed to service any unicast client requests for retransmission of specific messages. The listeners must be configured with these IP addresses and port combinations to which they can submit the requests.

3.2.1. Number Fields

All number fields in the MoldUDP64 messages specified in this document (i.e. sequence number, message counts and message lengths etc) are binary numbers formatted in Big Endian mode (i.e. most significant byte first).

Note: This need not apply to the data contained the Message Data fields of the Message Blocks.

3.2.2. Terms

Message

A message is an atomic piece of information carried by the MoldUDP64 protocol.

MoldUDP64 can theoretically handle individual messages from zero bytes up to 64KB in length although individual messages should be kept small enough so that the UDP underlying network protocol can efficiently carry the resulting MoldUDP64 packets.

The contents of a MoldUDP64 message are defined by the higher level application.

Session

A Session is a sequence of one or more messages.

While a single session can last indefinitely, typically the application will define a session to logically group messages together based on time delimitation.

Once a session is terminated, no more messages can be sent on that session. Depending on the design of the MoldUDP64 system and the application, receivers may still be able to re-request messages from a terminated session.

A session is considered active if it has started but not yet been terminated.

3.2.3. Downstream Packet

A MoldUDP64 transmitter sends “downstream” packets that are received by MoldUDP64 listeners. A MoldUDP64 packet may contain a payload of 0 or more data stream messages.

Each MoldUDP64 packet consists of a Downstream Packet Header and of a series of Message Blocks. The Message Blocks carry the actual data of the stream.

Downstream Packet Header

Name	Offset	Length	Value	Notes
Session	0	10	ANUM	Indicates the session to which this packet belongs
Sequence Number	10	8	NUM	The sequence number of the first message in the packet
Message Count	18	2	NUM	The count of messages contained in this packet

Sequence Number

The Sequence Number field of the packet Header indicates the sequence number of the first message in the packet. If there is more than one message contained in a packet, any messages following the first message are implicitly numbered sequentially.

Message Count

The number of Message Blocks contained in a MoldUDP64 packet is specified by the Message Count field of the Packet Header. The maximum payload size of a Downstream Packet is determined by the sender. Note that a Message Count of zero denotes a heartbeat and that a Message Count of 0xFFFF (hex, or 65535 in decimal) denotes end of session.

Message Block

The first field of a Message Block is the two byte Message Length. The remainder of the Message Block is the variable length Message Data field. The first Message Block field will always start immediately following the Header which is 20 bytes from the beginning of the packet. Subsequent Message Blocks will begin after the last byte of the previous Message Block.

Name	Offset	Length	Value	Notes
Message Length	*	2	NUM	Indicates the length in bytes of the message contained in this Message Block
Message Data	*	*	ANUM	The message data

* = Variable values

Message Length

The Message Length is an unsigned binary count representing the number of message data bytes following this Message Length field. The message length field value does not include the two bytes occupied by the message length field. The total size of the message block is the value of the message length field plus two.

Message Data

The Message Data is actual data of the message being transmitted by MoldUDP64. It is variable length and can be zero length. The meaning of the data is application specific.

Heartbeats

Heartbeats are sent periodically by the server so receivers can sense packet loss even during times of low traffic. Typically, these packets are transmitted once per second and contain the next expected Sequence Number.

A Heartbeat packet is a MoldUDP64 packet with a Message Count of zero.

End of Session

When the current session is complete, Downstream Packets are sent with a Message Count of 0xFFFF(hex, or 65535 in decimal) for a short while in place of Heartbeats. These Downstream Packets contain the next expected Sequence Number, just like Heartbeats.

While the End of Session messages persist, re-requests may be made on the current session. This is the last chance to ensure that all messages have been received.

Request Packet

The Request Packet is sent to request the retransmission of a particular message or group of messages. The request packet is sent to a Re-request server. A receiver may need to send this request when it detects a sequence number gap in received messages.

The response to a valid Request Packet is a standard Downstream Packet unicast back to the source of the retransmission request. This allows downstream MoldUDP64 users to read the retransmitted Downstream Packet in their multicast processing socket if the request was made from that socket (in other words, the client need only have one socket open to listen to the multicast and to process retransmissions, even though the retransmissions are not multicast).

Name	Offset	Length	Value	Notes
Session	0	10	ANUM	Indicates the session to which this packet belongs
Sequence Number	10	8	NUM	First requested sequence number
Requested Message Count	18	2	NUM	The number of messages requested for retransmission

Sequence Number

The Sequence Number field of the packet Header indicates the sequence number of the first message requested.

Requested Message Count

The Message Count indicates how many messages should be retransmitted. If the total size of the requested messages exceeds the maximum payload size of the of one UDP packet, only the number of messages that completely fit will be returned. Additional retransmission requests must be made for the subsequent messages if they are still desired.

3.2.4. Receiver Example

A typical MoldUDP64 receiver client would be configured with the following parameters:

- The UDP port to listen on and the Multicast group to join
- A list of one or more Request Servers that are available to answer retransmission requests for this stream. Each server is specified as a host IP address and a UDP port to which to send requests
- A session and sequence number of the next expected message if the client is being restarted

A typical MoldUDP64 receiver client might obey the following flowchart:

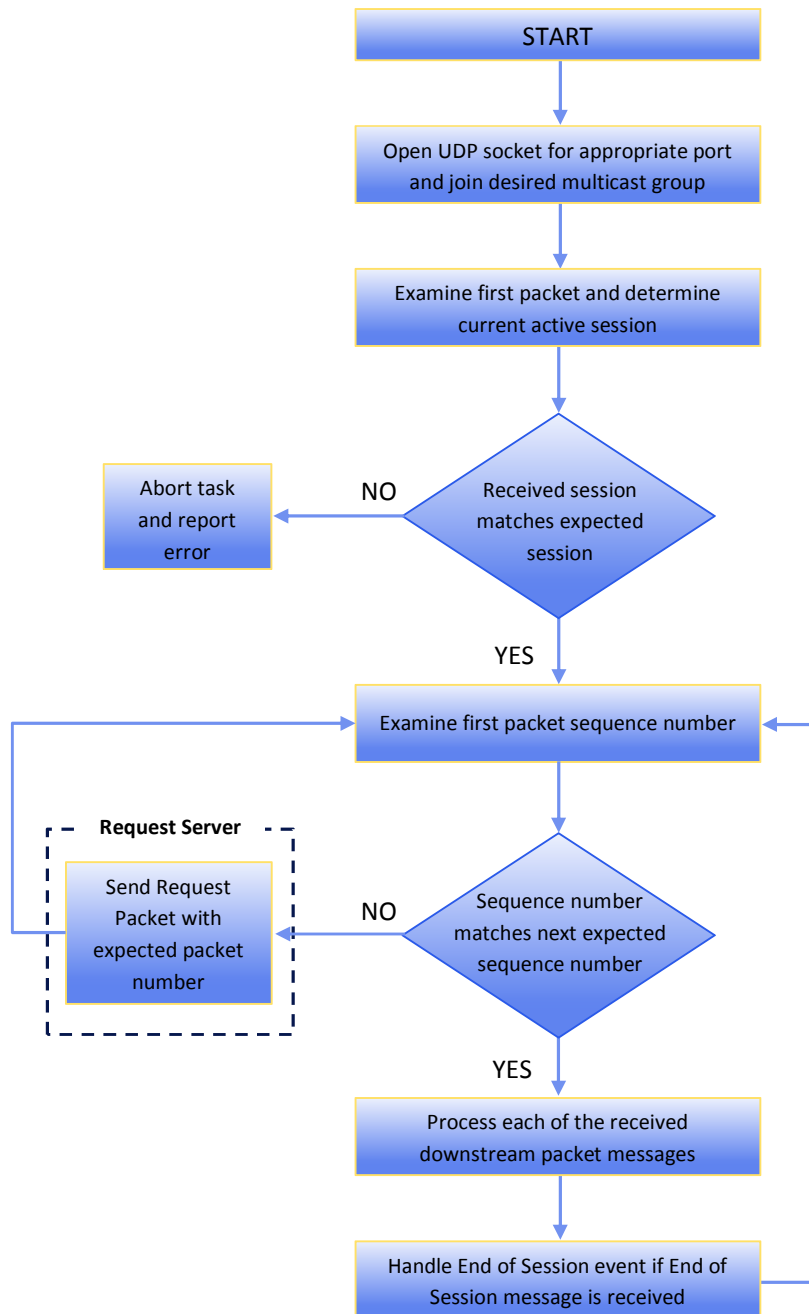


Figure 3.1 Client Receiver Flowchart

3.3. ITCH Protocol

The messages that make up the ITCH protocol are typically delivered using a higher level protocol that takes care of sequencing and delivery guarantees.

3.3.1. Data Types

All numeric fields are composed of binary encoded numbers.

All alpha fields are left justified and padded on the right with spaces, and are composed of non-control ISO 8859-1 (Latin-1) encoded bytes.

Type	Size	Notes
Numeric	1, 2, 4, or 8 bytes	Unsigned big-endian binary encoded numbers Note: Transport layers MoldUDP64 uses big-endian for their numeric values
Alpha	Variable	Left justified and padded on the right with spaces
Price	4 bytes	Prices are signed integer fields. Number of decimals is specified in the Order Book Directory message Note 1: A Price field with bit 31 set (the highest bit, MIN_INT) while all other bits are zero (decimal - 2147483648) means that no price was available. This value also represents a market order in the Add Order messages Note 2: The tick size table entry message contains an 8 Byte price field

3.3.2. Message Formats

The ITCH feed is composed of a series of messages that describe orders added to, removed from, and executed on the Trading system. It also contains messages for basic reference data of the Order Books as well as state changes and halts.

Message Type Reference Codes

The ITCH message reference protocol utilises the following code references:

Code	Message Type
A	Add Order
C	Order Executed
D	Order Delete
E	Order Executed
L	Tick Size
M	Combination Order Book Directory
O	Order Book State
P	Trade Message Identifier
R	Order Book Directory
S	System Event
T	Seconds
U	Order Replace
Z	Equilibrium Price Update

Time Messages

For bandwidth efficiency, the ITCH timestamp is separated into two parts:

Timestamp Portion	Message Type	Notes
Seconds	Standalone	Unix Time (number of seconds since the start of 1970-01-01 00:00:00 UTC) Note: A Timestamp - Second message will be disseminated for every second for which there is at least one payload message
Nanoseconds	Field within individual message	Reflects the number of nanoseconds since the most recent Timestamp - Seconds message that the data message was generated

Seconds Message

This message is sent every second for which at least one ITCH message is being generated. The message contains the number of seconds since the start of 1970-01-01 00:00:00 UTC, also called Unix Time.

Name	Offset	Length	Value	Notes
Message Type	0	1	"T"	T - Seconds Message
Second	1	4	Numeric	Unix time(number of seconds since the start of 1970-01-01 00:00:00 UTC)

3.3.3. Reference Data Messages

Order Book Directory

At the start of each trading day, Order Book directory messages are disseminated for all active instruments, including halted instrument, in the Trading system.

Note: Intra-day transmissions of this message may occur when new Order Books are added to the system. Updates to existing Order Books may also be represented by intra-day Order Book Directory messages.

Name	Offset	Length	Value	Notes
1 - Message Type	0	1	"R"	R - Order Book Directory Message
2 - Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
3 - Order Book ID	5	4	Numeric	Denotes the primary identifier of an Order Book Note: Expired Order Book ID's may be reused for new instruments
4 - Symbol	9	32	Alpha	Instrument short name
5 - Long Name	41	32	Alpha	Human readable long name of Instrument series
6 - Reserved	73	12		
7 - Financial Product	85	1	Numeric	Values: 1 = Option 2 = Forward 3 = Future 4 = FRA 5 = Cash 6 = Payment 7 = Exchange Rate 8 = Interest Rate Swap 9 = REPO 10 = Synthetic Box Leg/Reference 11 = Standard Combination 12 = Guarantee 13 = OTC General 14 = Equity Warrant 15 = Security Lending
8 - Trading Currency	86	3	Alpha	Trading Currency
9 - Number of decimals in Price	89	2	Numeric	This value defines the number of decimals used in price for this Order Book Note: A value of 256 means that the instrument is traded in fractions (each fraction is 1/256)
10 - Number of decimals in	91	2	Numeric	This value defines the number of decimals in Nominal Value

Nominal Value				
Name	Offset	Length	Value	Notes
11 - Reserved	93	4		
12 - Reserved	97	4		
13 - Reserved	101	4		
14 - Nominal Value	105	8	Numeric	Nominal Value

Order Book Directory Example Message

R,2015-06-14T17:28:06.175816597(1434302886175816597),528394,CYM15,,,3,USD,4,0,0,1,0,0

Note: The following example is a breakdown of the Order Book Directory Message identified above, illustrating the alphaneumerical message portions.

1	2	3	4						
R,	2015-06-14T17:28:06.175816597(1434302886175816597),	528394,	CYM15,						
5	6	7	8	9	10	11	12	13	14
,	,	3,	USD,	4,	0,	0,	1,	0,	0

Combination Order Book Directory

The Combination Order Book Directory is a specialized directory message used when Combination Order Books are traded at the marketplace, representing both standard combinations defined by the exchange.

Note: Intra-day transmissions of this message may occur when new Combination Order Books are added to the system. This is typically the case for exchange listed. Updates to existing combination Order Books may also be represented by intra-day Combination Order. Book

Name	Offset	Length	Value	Notes
Message Type	0	1	"M"	M - Combination Order Book Directory Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order Book ID	5	4	Numeric	Denotes the primary identifier of an Order Book Note: Expired Order Book ID's may be reused for new instruments
Symbol	9	32	Alpha	Instrument series short name
Long Name	41	32	Alpha	Human readable long name of Instrument series
Reserved	73	12		

Name	Offset	Length	Value	Notes
Financial Product	85	1	Numeric	Values: 1 = Option 2 = Forward 3 = Future 4 = FRA 5 = Cash 6 = Payment 7 = Exchange Rate 8 = Interest Rate Swap 9 = REPO 10 = Synthetic Box Leg/Reference 11 = Standard Combination 12 = Guarantee 13 = OTC General 14 = Equity Warrant 15 = Security Lending
Trading Currency	86	3	Alpha	Trading Currency
Number of decimals in Price	89	2	Numeric	This value defines the number of decimals used in price for this Order Book Note: A value of 256 means that the instrument is traded in fractions (each fraction is 1/256)
Number of decimals in Nominal Value	91	2	Numeric	This value defines the number of decimals in Nominal Value
Reserved	93	4		
Reserved	97	4		
Reserved	101	4		
Nominal Value	105	8	Numeric	Nominal Value
Leg 1, Symbol	113	32	Alpha	Leg Symbol
Leg 1, Side	145	1	Alpha	Values: B = As Defined C = Opposite
Leg 1, Ratio	146	4	Numeric	
Leg 2, Symbol	150	32	Alpha	Leg Symbol
Leg 2, Side	182	1	Alpha	Values: B = As Defined C = Opposite
Leg 2, Ratio	183	4	Numeric	
Leg 3, Symbol	187	32	Alpha	Leg Symbol
Leg 3, Side	219	1	Alpha	Values: B = As Defined C = Opposite
Leg 3, Ratio	220	4	Numeric	

ITCH

Leg 4, Symbol	224	32	Alpha	Leg Symbol
---------------	-----	----	-------	------------

Name	Offset	Length	Value	Notes
Leg 4, Side	256	1	Alpha	Values: B = As Defined C = Opposite
Leg 4, Ratio	257	4	Numeric	

Combination Order Book Directory Example

M,2015-06-14T17:28:06.175816597(1434302886175816597),4984842,CYM15-CYH18,,,11,USD,4,0,0,1,0,0,CYM15,C,1,CYH18,B,1,,?,0,,?,0

Tick Size Table Entry

This message contains information on a tick size for a price range. Together, all Tick Size messages with the same Book ID form a complete Tick Size Table. Each Order Book has a set of Tick Size Table Entries to define its tick size table.

Note: The number of decimals in prices are given by the Order Book Directory message for this Order Book.

Name	Offset	Length	Value	Notes
Message Type	0	1	"L"	L - Tick Size Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order Book ID	5	4	Numeric	The Order Book this entry belongs to
Tick Size	9	8	Numeric	Tick Size for the given price range
Price From	17	4	Price	Start of price range for this entry
Price To	21	4	Price	End of price range for this entry, zero (0) means infinity

Tick Size Table Entry Example Message

L,2015-06-14T17:28:06.175816597(1434302886175816597),CYM15(528394),1,1,9999990000

3.3.4. Event & State Change Messages

System Event

The system event message type is used to signal a market or data feed handler event.

Name	Offset	Length	Value	Notes
Message Type	0	1	"S"	S - System Event Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Event Code	5	1	Alpha	See System Event Codes below

The system supports the following event codes on a daily basis.

Code	Explanation
"O"	Start of Messages Outside of time stamp messages, the start of day message is the first message sent in any trading day
"C"	End of Messages This is always the last message sent in any trading day

Order Book State

The Order Book State Message relays information on state changes.

Name	Offset	Length	Value	Notes
Message Type	0	1	"O"	O - Order Book State Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order Book ID	5	4	Numeric	The Order Book ID
State Name	9	20	Alpha	Name of Order Book State

Order Book State Example Message

O,2015-06-14T17:28:06.175816597(1434302886175816597),CYM15(528394),CLOSE

3.3.5. Market by Order Messages

Order ID's are only unique per Order Book and side. When modifying or deleting orders, be careful to only update the order with the correct side and Order Book, since the same Order ID may be present in multiple Order Books and/or sides.

Add Order

An Add Order Message indicates that a new order has been accepted by the Trading system and was added to the displayable book. The message includes an Order ID that is unique per Order book and side used by Trading to track the order.

Note: Refer to the Data types chapter for a description of how market orders are represented.

Name	Offset	Length	Value	Notes
Message Type	0	1	"A"	A - Add Order Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order ID	5	8	Numeric	The identifier assigned to the new order Note that the number is only unique per Order Book and side
Order Book ID	13	4	Numeric	The Order Book ID
Side	17	1	Alpha	The type of order being added: B = Buy order S = Sell order
Order Book Position	18	4	Numeric	Rank within the Order Book
Quantity	22	8	Numeric	The visible quantity of the order Note: Orders with an undisclosed quantity will have this field set to 0
Price	30	4	Price	The display price of the new order
Order Attributes	34	2	Numeric	Additional order attributes Values: 0 = Not applicable 8192 = Bait/implied order Note 1: Application types may be defined by the marketplace Note 2: This field is a bitmap. Multiple values may be set simultaneously
Lot Type	36	1	Numeric	Lot Type Values: 2 = Round Lot

Add Order Example Message

A,2015-06-16T08:34:11.709583662(1434443651709583662),6571328004721632139,NUM16(262374),B,1,50,2038000,0,2

Modify Order

Modify Order messages always include the Order ID, Order Book ID and Side of the Add Order to which the update applies.

To determine the current display quantity for an order, ITCH subscribers must deduct the number of lots stated in the Modify message from the original number of lots stated in the Add Order message with the same Order ID.

The Trading system may send multiple Modify Order messages for the same order and the effects are cumulative. When the number of display lots for an order reaches zero, the order is dead and should be removed from the book.

Order Executed

This message is sent whenever an order on the book is executed in whole or in part. If the incoming order causing the match cannot be fully filled, the remainder will be placed in the book after the match has occurred.

It is possible to receive several Order Executed Messages for the same order if that order is executed in several parts. Multiple Order Executed Messages on the same order are cumulative.

Name	Offset	Length	Value	Notes
Message Type	0	1	"E"	E - Order Executed Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order ID	5	8	Numeric	The order ID is associated with the executed order
Order Book ID	13	4	Numeric	The Order Book ID
Side	17	1	Alpha	The type of order being added Values: B = Buy order S = Sell order
Executed Quantity	18	8	Numeric	The quantity being executed
Match ID	26	8	Numeric	Assigned by the system to each match executed
Combo Group ID	34	4	Numeric	Used to group Combination Order Book executions and the trades in the constituent Order Books together
Reserved	38	7		
Reserved	45	7		

Order Executed Example Message

```
E,2015-06-16T08:36:40.035127116(1434443800035127116),6571328004721632139,NUM16
(262374) ,B,10,657132749362036751,0,,
```

Order Executed with Price

This message is sent in the relatively rare event when an order on the book is executed in whole or in part with a price different than the initial display price. If the incoming order causing the match cannot be fully filled, the remainder will be placed in the book after the match has occurred.

It is possible to receive several Order Executed Messages for the same order if that order is executed in several parts. Multiple Order Executed Messages on the same order are cumulative.

The executions may be marked as non-printable. If a participant is looking to use the ITCH data in trade tickers or volume calculations, SGX recommends that participants ignore messages marked as non-printable to prevent double counting.

Note 1: Combination Orders on the book that execute will always be represented by this message.

Note 2: Combination Orders that execute will have the Printable flag set to "N". The trades that occur in the legs of the combination will be printable. This avoids double counting of the Combination Order and its leg trades. Leg trades will be published with the Trade message.

Name	Offset	Length	Value	Notes
Message Type	0	1	"C"	C - Order Executed Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order ID	5	8	Numeric	The order ID is associated with the executed order
Order Book ID	13	4	Numeric	The Order Book ID
Side	17	1	Alpha	The type of order being added: B = Buy order S = Sell order
Executed Quantity	18	8	Numeric	The quantity being executed
Match ID	26	8	Numeric	Assigned by the system to each match executed
Combo Group ID	34	4	Numeric	Used to group Combination Order group executions and the trades in the constituent orderbooks together
Reserved	38	7		
Reserved	45	7		
Trade Price	52	4	Price	
Occurred at Cross	56	1	Alpha	Values: Y = Yes, trade occurred at the cross N = No, trade occurred at continuous market
Printable	57	1	Alpha	Indicates if the trade should be included in trade tickers and volume calculations Values: Y = Printable N = Non-printable

Order Executed With Price Example Message

C,2015-06-22T16:11:46.707568156(1434989506707568156),6573775517605104436,ELH16-ELH17
(36110438),B,100,657377390699228928,12032,,,4100,N,N

Order Replace

This message is sent whenever an order on the book has been replaced. The remaining quantity from the original order is no longer accessible, and must be removed.

The Side, and Order Book ID, remain the same as the original order.

Name	Offset	Length	Value	Notes
Message Type	0	1	"U"	U - Order Replace Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order ID	5	8	Numeric	The original order identifier of the order being replaced The Order ID is only unique per Order Book and side Note: The Order ID does not change when the order is replaced
Order Book ID	13	4	Numeric	The Order Book ID
Side	17	1	Alpha	The type of order being added: B = Buy order S = Sell order
New Order Book Position	18	4	Numeric	New Rank within Order Book
Quantity	22	8	Numeric	The new visible quantity of the order
Price	30	4	Price	The new Price of the order
Order Attributes	34	2	Numeric	Additional order attributes Values: 0 = Not applicable 8192 = Bait/implied order Note1: Applicable types may be defined by the marketplace Note 2: This field is a bit map. Multiple values may be set simultaneously

Order Replace Example Message

Note: There will be no Order Replace Message seen in ITCH Feed.
For Every Order Update Message via OUCH, there is an "Order Delete Message" followed by an "ADD Order Message in ITCH".

Order Delete

This message is sent whenever an order on the book is being deleted. There will be no remaining quantity, so the order should be removed from the book.

Please note that normally no Order Delete message is sent when an order is completely filled. The receiver needs to keep track of the remaining quantity on all orders by recalculating the remaining quantity on each Order Executed message received.

Orders must be removed from the book when remaining quantity reaches 0.

Note: Order Deletes are sent out when orders are suspended due to connection loss.

Name	Offset	Length	Value	Notes
Message Type	0	1	"D"	D - Order Delete Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order ID	5	8	Numeric	The ID of the order being deleted Note: The Order ID is only unique per Order Book and side
Order Book ID	13	4	Numeric	The Order Book ID
Side	17	1	Alpha	The type of order being added Values: B = Buy order S = Sell order

Order Delete Example Message

D,2015-06-18T14:43:32.152448316(1434638612152448316),6572174353797114250,SGM15(15204552),S

3.3.6. Trade Messages

Trade

The Trade Message is designed to provide execution details for normal match events involving non-displayable order types. This message is also used to publish individual cross trades. Since no Add Order Message is generated when a non-displayed order is initially received, the Order Executed message cannot be used for all matches. The Trade Message is used to report a match for a non-displayable order in the book.

It is possible to receive multiple Trade Messages for the same order if that order is executed in several parts. Trade Messages for the same order are cumulative. Trade Messages should be included in trade tickers as well as volume and other market statistics. Since Trade Messages do not affect the displayed book, however, they may be ignored by participants just looking to build and track the Order Book review.

Name	Offset	Length	Value	Notes
Message Type	0	1	"P"	P - Trade Message Identifier
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Match ID	5	8	Numeric	Assigned by the system to each match executed
Combo Group ID	13	4	Numeric	Used to group Combination Order Book executions and the trades in the constituent Order Books together
Side	17	1	Alpha	The type of non-display on the book being matched B = Buy order S = Sell order Note: Will be set to blank (space) for anonymous markets
Quantity	18	8	Numeric	The quantity being matched in this execution
Order Book ID	26	4	Numeric	The Order Book ID
Trade Price	30	4	Price	
Reserved	34	7		
Reserved	41	7		
Printable	48	1	Alpha	Indicates if the trade should be included in trade tickers and volume calculations Values: Y = printable N = non-printable
Occurred at Cross	49	1	Alpha	Values: Y = Trade occurred at the cross N = Trade occurred at continuous market

Trade Example Message

P,2015-06-22T16:11:46.707568156(1434989506707568156),657377390699228930,12032,,100,ELH16
(15532134),1002100,,,Y,N

3.3.7. Auction Messages

Markets by order dissemination may be disabled during auctions by configuration. In such cases, every existing order will be removed from the book by an Order Delete message immediately prior to the auction.

Note: The owners of these orders must not interpret this as order cancellations. Use the private order flow to determine the state of your orders.

Equilibrium Price Update

This message is used if/when auctions occur. The message provides the changes in equilibrium price.

Note: If any Price field has bit 31 set (the highest bit, MIN_INT) while all other bits are zero (decimal - 2147483648), this means that no price is available.

Name	Offset	Length	Value	Notes
Message Type	0	1	"Z"	Z - Equilibrium Price Update Message
Timestamp - Nanoseconds	1	4	Numeric	Nanoseconds portion of the timestamp
Order Book ID	5	4	Numeric	The Order Book ID
Available Bid Quantity at Equilibrium Price	9	8	Numeric	Quantity at equilibrium price on the bid side
Available Ask Quantity at Equilibrium Price	17	8	Numeric	Quantity at equilibrium price on the bid side
Equilibrium Price	25	4	Price	Equilibrium Price
Best Bid Price	29	4	Price	Best Bid Price
Best Ask Price	33	4	Price	Best Ask Price
Best Bid Quantity	37	8	Numeric	Best Bid Quantity
Best Ask Quantity	45	8	Numeric	Best Ask Quantity

Equilibrium Price Update Example Message

Z,2015-06-22T12:02:47.687979528(1434974567687979528),ELH16-ELH17(36110438),50,30,600,
600 ,500,50,30

3.3.8. Building an Order Book View

The information needed to build an order book view from the ITCH message flow is contained in the following Add and the Modify Order Messages sections:

- Add Order
- Order Executed
- Order Executed with Price
- Order Replace
- Order Delete

Orders shall be ranked by:

- **Order book Position**
“1” denotes the highest ranked order. For an Order Replace, the order must be removed from its previous position and inserted at New Order book Position. An order inserted at an existing position shifts the order on that position down (and all orders below as well. A deleted or fully filled order causes existing orders below it to shift their position up one step to fill the “void”

The Order Executed (with Price) message signals a partial or full fill. The order quantity must be reduced by the quantity of the Order Executed message.

The Order Replace message signals that the order has been modified. The current rank may or may not be lost in the process. Order book Position will show the new rank within the book.

The Order Delete message tells the recipient to remove the order referenced.

3.3.9. Building a Trade Ticker

The information needed to build a Trade Ticker is contained in the following messages sections:

- Order Executed
- Order Executed with Price
- Trade

Note: Reported trades are not included in ITCH.

4. Glimpse

4.1. Overview

A complement to the SGX Titan ITCH real-time data feed product, Glimpse is a point-to-point data feed connection that provides direct data feed customers with a snapshot of the current state of the order book traded in the SGX Titan.

Glimpse utilises all message formats used by ITCH to describe the current state of the book, with the exception of messages that change the state of the Order Book i.e Delete, Execute and Trade.

The following Market information data messages are featured:

Data Elements	Description
Reference Data	Order Book Directory messages for all traded Order Books
	Tick Size Table Entry messages to convey Tick Sizes tables used for each Order Book
Event Controls	Order Book State messages with the current state of all Order Books
	Add order messages for all displayable orders in the system at the time of login
Glimpse Snapshot	Snapshot message that gives the SoupBinTCP sequence number to be used when reconnecting to the ITCH feed

4.2. SoupBinTCP Protocol

SoupBinTCP is a lightweight point-to-point protocol, built on top of TCP/IP sockets that allow delivery of a set of sequenced messages from a server to a client in real-time. SoupBinTCP guarantees that the client receives each message generated by the server in sequence, even across underlying TCP/IP socket connection failures.

SoupBinTCP clients can send messages to the server. These messages are not sequenced and may be lost in the case of a TCP/IP socket failure.

SoupBinTCP is ideal for systems where a server needs to deliver a logical stream of sequenced messages to a client in real-time but does not require the same level of guarantees for client generated messages either because the data stream is unidirectional or because the server application generates higher-level sequenced acknowledgments for any important client-generated messages.

SoupBinTCP is designed to be used in conjunction with higher level protocols that specify the contents of the messages that SoupBinTCP messages deliver. The SoupBinTCP protocol layer is opaque to the higher-level messages.

There is an authorisation requirement for ITCH/Glimpse.

SoupBinTCP Logical Packets

The SoupBinTCP client and server communicate by exchanging a series of logical packets.

Each SoupBinTCP logical packet has:

- A two byte big-endian length that indicates the length of rest of the packet (meaning the length of the payload plus the length of the packet type - which is 1)
- A single byte header which indicates the packet type
- A variable length payload

SoupBinTCP Logical Packet Structure

Two Byte Packet Length	Packet Type	Variable-length payload
------------------------	-------------	-------------------------

Note 1: The SoupBinTCP logical packets does not necessarily map directly to physical packets on the underlying network socket; they may be broken apart or aggregated by the TCP/IP stack

Note 2: The SoupBinTCP protocol does not define a maximum payload length

Protocol Flow

A SoupBinTCP connection begins with the client opening a TCP/IP socket to the server and sending a Login Request Packet. If the login request is valid, the server responds with a Login Accepted Packet and begins sending Sequenced Data Packets. The connection continues until the TCP/IP socket is broken.

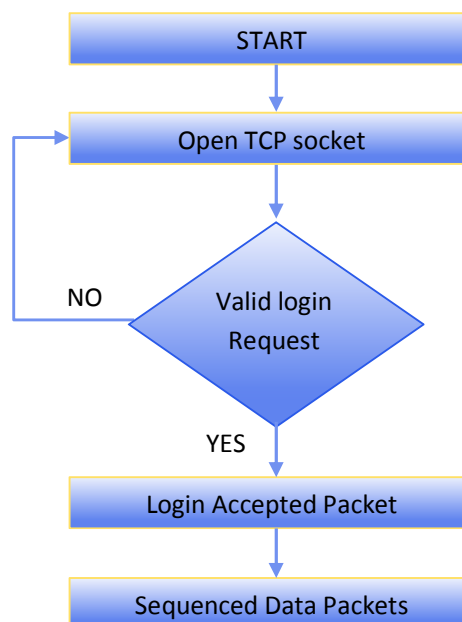


Figure 4.1 SoupBinTCP Protocol Flow

Each Sequenced Data Packet carries a single higher-level protocol message. Sequenced Data Packets do not contain an explicit sequence number; instead both client and server compute the sequence number locally by counting messages as they go. The sequence number of the first sequenced message in each session is always 1.

Typically, when initially logging into a server, the client will set the Requested Sequence Number field to 1 and leave the Requested Session field blank in the Login Request Packet. The client will then inspect the Login Accepted Packet to determine the currently active session. Starting at 1, the client begins incrementing its local sequence number each time a Sequenced Data Packet is received.

If the TCP/IP connection is ever broken, the client can then re-log into the server indicating the current session and its next expected sequence number. By doing this, the client is guaranteed to always receive every sequenced message in order, despite TCP/IP connection failures.

SoupBinTCP also permits the client to send messages to the server using Unsequenced Data Packets at any time after the Login Accepted Packet is received. These messages may be lost during TCP/IP socket connection failures.

Heartbeats

SoupBinTCP uses logical heartbeat packets to quickly detect link failures. The server must send a Server Heartbeat packet anytime more than 1 second has past since the server last sent any data. This ensures that the client will receive data on a regular basis.

If the client does not receive anything (neither data nor heartbeats) for an extended period of time, it can assume that the link is down and attempt to reconnect using a new TCP/IP socket.

Similarly, once logged in, the client must send a Client Heartbeat packet anytime more than 1 second has past since the client last sent anything. If the server doesn't receive anything from the client for an extended period of time (typically 15 seconds), it can close the existing socket and listen for a new connection.

End of Session Marker

The server indicates that the current session has terminated by sending an End of Session Message. This indicates that there will be no more messages contained in this session.

The client will have to reconnect and login with the new Session ID or a blank (space-filled) Session ID to begin receiving messages for the next available session.

Data Types

Character data fields are standard ASCII bytes. Integer fields are binary big-endian values.

4.2.1. SoupBinTCP Packet Types

Logical Packets Sent by a SoupBinTCP Server

Code	Packet Type
A	Login Accepted
H	Server Heartbeat
J	Login Rejected
S	Sequenced Data Packed
Z	End of Session

Logical Packets Sent by a SoupBinTCP Client

Code	Packet Type
L	Login Request
O	Logout Request
R	Client Heartbeat
U	Unsequenced Data Packed

Logical Packets Sent by a SoupBinTCP Server

Login Accepted Packet

The SoupBinTCP server sends a Login Accepted Packet in response to receiving a valid Login Request from the client. This packet will always be the first non-debug packet sent by the server after a successful login request.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"A"	Login Accepted Packet
Session	3	10	Alpha-numeric	The session ID of the session that is now logged into. Left padded with spaces
Sequence Number	13	20	Numeric	The sequence number in ASCII of the next Sequenced Message to be sent. Left padded with spaces

Login Rejected Packet

The SoupBinTCP server sends this packet in response to an invalid Login Request Packet from the client. The server closes the socket connection after sending the Login Reject Packet. The Login Rejected Packet will be the only non-debug packet sent by the server in the case of an unsuccessful login attempt.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"J"	Login Rejected Packet
Reject Reason Code	3	1	Alpha	See Login Reject Codes below

Login Reject Codes

Name	Explanation
"A"	Not Authorized. There was an invalid username and password combination in the Login Request Message
"S"	Session not available. The Requested Session in the Login Request Packet was either invalid or not available

Sequenced Data Packet

The Sequenced Data Packets act as an envelope to carry the actual sequenced data messages that are transferred from the server to the client. Each Sequenced Data Packet carries one message from the higher-lever protocol.

The sequence number of each message is implied; the initial sequence number of the first Sequenced Data Packet for a given TCP/IP connection is specified in the Login Accepted Packet and the sequence number increments by 1 for each Sequenced Data Packet transmitted.

Since SoupBinTCP logical packets are carried via TCP/IP sockets, the only way logical packets can be lost is in the event of a TCP/IP socket connection failure. In this case, the client can reconnect to the server and request the next expect sequence number and pick up where it left off.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"S"	Sequenced Data Packed
Message	3	Variable	Any	Defined by a higher-level protocol, but must not contain any embedded linefeeds. A Message with zero length is a special End of Session marker that indicated that there are no more messages available in this session

Server Heartbeat Packet

The server should send a Server Heartbeat Packet anytime more than 1 second passes where no data has been sent to the client. The client can then assume that the link is lost if it does not receive anything for an extended period of time.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"H"	Server Heartbeat Packet

End of Session Packet

The server will send an End of Session Packet to denote that the current session is finished. The connection will be closed shortly after this packet, and the user will no longer be able to reconnect to the current session.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"Z"	End of Session Packet

Logical Packets sent by the SoupBinTCP Client

Login Request Packet

The SoupBinTCP client must send a Login Request Packet immediately upon establishing a new TCP/IP socket connection to the server.

Client and server must have mutually agreed upon the username and password fields. They provide simple authentication to prevent a client from inadvertently connecting to the wrong server.

Both Username and Password are case-insensitive and should be padded on the right with spaces.

The server can terminate an incoming TCP/IP socket if it does not receive a Login Request Packet within a reasonable period of time (typically 30 seconds).

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"L"	Login Request Packet
Username	3	6	Alphanumeric	Username
Password	9	10	Alphanumeric	Password
Requested Session	19	10	Alphanumeric	Specifies the session the client would like to log into, or all blanks to log into the currently active session
Requested Sequence Number	29	20	Numeric	Specifies the next sequence number in ASCII the client wants to receive upon connection, or 0 to start receiving the most recently generated message

Unsequenced Data Packets

The Unsequenced Data Packets act as an envelope to carry the actual data messages that are transferred from the client to the server. These messages are not sequenced and may be lost in the event of a socket failure. The higher-level protocol must be able to handle these lost messages in the case of a TCP/IP socket connection failure.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"U"	Unsequenced Data Packed
Message	3	Variable	Alpha-numeric	Defined by a higher-level protocol, but must not contain any embedded linefeeds

Client Heartbeat Packets

The client should send a Client Heartbeat Packet anytime more than 1 second passes where no data has been sent to the server. The server can then assume that the link is lost if it does not receive anything for an extended period of time.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Integer	Number of bytes after this field until the next packet
Packet Type	2	1	"R"	Client Heartbeat Packet

Logout Request Packet

The client may send a Logout Request Packet to request the connection be terminated. Upon receiving a Logout Request Packet, the server will immediately terminate the connection and close the associated TCP/IP socket.

Name	Offset	Length	Value	Notes
Packet Length	0	2	Binary	Number of bytes after this field until the next packet
Packet Type	2	1	"O"	Logout Request Packet

4.2.2. Data Types

Glimpse messages have the same message types as ITCH messages.

4.2.3. Glimpse Messages

Glimpse uses a subset of ITCH messages.

Glimpse uses all messages used by ITCH to describe the current price of the book, with the exception of those messages that change the picture of the Order Book (e.g Delete, Execute and Trade).

End of Snapshot Message

The end of snapshot message returns the current ITCH sequence number to be used when connecting to the ITCH feed. To maintain a real-time order display, firms should begin to process the real-time ITCH messages beginning with the sequence number stated in this snapshot message.

Name	Offset	Length	Value	Notes
Message Type	0	1	"G"	G - End of Snapshot Message
Sequence Number	1	20	Alpha	ITCH SoupBinTCP sequence number when the snapshot was taken. To be used when logging in to the SoupBinTCP ITCH feed Note: While Glimpse is a binary feed, the SoupBinTCP uses ASCII characters to represent the sequence number

5. Configuration & Infrastructure

Note: The Configuration and Infrastructure data is under development and will be available for implementation at a later date.

6. Appendix 1 - ITCH Scenario Examples

To illustrate ITCH functionality for both common and ad-hoc scenarios, the following examples are provided.

Order Types	Scenario
Disclosed Orders, Continuous Trading	New Order Entry, Amend & Delete
	New Order Fully Trades with Existing Order(s)
	New Order Partially Trades with Existing Order(s)
	Existing Order is Amended and Immediately Fully Trades with Opposing Order
	Existing Order is Amended and Immediately Partially Trades with Opposing Order
Disclosed Order Execution, Auctions	Orders Trade in Auction
Instrument Creation	Standard Instrument Creation
Miscellaneous Tasks	Customer to Customer Order Transfer

6.1. Disclosed Orders, Continuous Trading

6.1.1. New Order Entry, Amend & Delete

Step	Scenario	ITCH Message	Attributes	Activity
1	Order A entered into book	A	Order ID, quantity and price	Store Order ID, quantity and price for Order A
2	Order A amended	U	Order ID, quantity and price	Modify quantity and price for Order A
3	Order A deleted	D	Order ID	Remove Order A

6.1.2. New Order Fully Trades with Existing Order(s)

Step	Scenario	ITCH Message	Attributes	Activity
1	Order A entered into book	A	Order ID, quantity and price of Order A	Store Order ID, quantity and price for Order A
2	New Order B is entered and fully trades with Order A (and possibly other existing orders)	E	Order ID and executed quantity for Order A	Subtract executed quantity from stored Order A. If stored Order A quantity is zero, delete the stored order

6.1.3. New Order Partially Trades with Existing Order(s)

Step	Scenario	ITCH Message	Attributes	Activity
1	Order A entered into book	A	Order ID, quantity and price of Order A	Store Order ID, quantity and price of Order A
2	New Order B is entered and partially trades with Order. Order A is fully executed	E	Order ID and executed quantity for Order A	Subtract executed quantity from stored Order A. Order A has fully traded, therefore the quantity results in zero. Delete stored Order A
3		A	Residual Order ID, quantity and price of Order B	Store Order ID, quantity and price for Order B

6.1.4. Existing Order is Amended and Immediately Fully Trades with Opposing Order

Step	Scenario	ITCH Message	Attributes	Activity
1	Order A entered into book	A	Order ID, quantity and price of Order A	Store Order ID, quantity and price of Order A
2	Order B entered into book	A	Order ID, quantity and price of Order B	Store Order ID, quantity and price of Order B
3	Order B amended to trade with Order A (and possibly other existing orders)	D	Order ID of Order B	Remove Order B
4		E	Order ID and executed quantity for Order A	Subtract executed quantity from stored Order A. Order A has fully traded, therefore the quantity results in zero. Delete stored order A

6.1.5. Existing Order is Amended and Immediately Partially Trades with Opposing Order

Step	Scenario	ITCH Message	Attributes	Activity
1	Order A entered into book	A	Order ID, quantity and price of Order A	Store Order ID, quantity and price of Order A
2	Order B entered into book	A	Order ID, quantity and price of Order B	Store Order ID, quantity and price of Order B
3	Order B amended to trade with Order A. Order A fully trades	D	Order ID of Order B	Remove Order ID B
4		E	Order ID and executed quantity for Order A	Subtract executed quantity from stored Order A. Order A has fully traded, therefore the quantity results in zero. Delete stored order A
5		A	Residual quantity and price of Order B	Store Order ID, quantity and price of Order B. Order ID will be the same as original Order B

6.2. Disclosed Order Execution, Auctions

6.2.1. Orders Trade in Auction

Step	Scenario	ITCH Message	Attributes	Activity
1	Instrument series in PRE_OPEN	O	Instrument series PRE_OPEN	Market is in PRE_OPEN. Session state change for each applicable instrument series
2	New Sell Order A entered into book	A	Order ID, quantity and price of Order A	Store Order ID, quantity and price of Order A
3		Z	Equilibrium price message for instrument series if applicable	Equilibrium message indicates auction price and surplus volume if applicable
4	New Buy Order B entered into book	A	Order ID, quantity and price of Order B	Store Order ID, quantity and price of Order B
5		Z	Equilibrium price message for instrument series if applicable	Equilibrium message indicates auction price and surplus volume if applicable
6	Instrument series in OPEN	O	Instrument series OPEN	Market opens. Session state change for each applicable instrument series
7		C	Executed quantity and price of Order A. Printable is set to Y (on Sell side)	Subtract executed quantity from stored Order A. If stored Order A quantity is zero, delete the stored order
8		C	Executed quantity and price of Order B. Printable is set to N (on Buy side)	Subtract executed quantity from stored Order B

6.3. Instrument Creation

6.3.1. Standard Instrument Creation

Step	Scenario	ITCH Message	Attributes	Activity
1	Instrument created	R	Instrument name XXX	Instrument name and Order Book ID provided
2		L	XXX tick table entry	The tick table definition for the instrument
3		O	Instrument series session state	Indicates current session state for instrument series

6.4. Miscellaneous Tasks

6.4.1. Customer to Customer Order Transfer

Step	Scenario	ITCH Message	Attributes	Activity
1	Order A entered into book	A	Order ID of Order A. Order book position 1	Store Order ID and order book position
2	Customer to customer Order Transfer from MMM to NNN	D	Order A	Delete the order from order book
3		A	Order ID of Order A. Order book position 1	Store Order ID and order book position

Singapore Exchange
London • Tokyo • Beijing
• Hong Kong • Mumbai

2 Shenton Way
#02-02 SGX Centre 1
Singapore 068804

main: +65 6236 8888
sgx.com